

Embedded System Software 과제 3

(과제 수행 결과 보고서)

과목명: [CSE4116] 임베디드시스템소프트웨어

담당교수: 서강대학교 컴퓨터공학과 박 성 용

학번 및 이름: 20181688, 조태연

개발기간: 2024. 05. 20. -2024. 06. 02.

최 종 보 고 서

I. 개발 목표

Module programming, 디바이스 드라이버 구현, interrupt 등, 배운 내용을 활용하여 간단한 stopwatch 프로그램을 작성한다.

II. 개발 범위 및 내용

가. 개발 범위

- 디바이스 드라이버와 interrupt 기능을 구현하여 stopwatch를 구현
- stopwatch를 사용하는 응용 프로그램

나. 개발 내용

-timer_drvier.c

디바이스 등록 및 등록해제와 stopwatch기능을 구현했고 이를 workqueue를 이용하여 workqueue에 작업을 등록하는 top half routine과 등록된 작업을 수행하는 bottom half routine으로 나누어서 top half intterup와 bottom half intterupt를 구현하였다.

-device.c

Bottom half routine에서 불리는 디바이스에 시간을 출력하는 함수들을 구현했다.

III. 추진 일정 및 개발 방법

가. 추진 일정

- 05.20~05.24 관련 내용학습
- 05.25 device.c 작성
- 05.26~05.28 timer_device.c의 stopwatch구현
- 06.01 intterupt구현 후 최종 디버깅 및 테스트
- 06.02 보고서 작성

나. 개발 방법

-timer_driver.c

```
//종료를 위해 vol_down버튼이 눌렸는지를 체크하는 변수들
int pause;
int check_voldown_press;

//시간 표현하기 위한 변수들
unsigned long current_time; //현재 시간
unsigned long print_time; //출력할 시간
unsigned long voldown_time; //voldown이 눌린 시간
unsigned long pause_time; //pause가 눌린 시간
unsigned long acc_pause_time; //pause가 쌓인 시간

//stopwatch의 현재상태
MODE mode;

//응용프로그램을 sleep상태로 만들기 위한 waitq
wait_queue_head_t waitq;
DECLARE_WAIT_QUEUE_HEAD(waitq);
//workqueue 생성을 위한 my_wq
static struct workqueue_struct *my_wq;
```

Interrupt와 stopwatch를 구현하기 위한 변수들이다.

```
//응용 프로그램을 waitq에서 sleep 시키기 위해 쓰임
static int device_write(struct file *filp, const char __user *buff, size_t len, loff_t *off){
    interruptible_sleep_on(&waitq);
    return SUCCESS;
}
```

응용프로그램을 sleep시키고 후에 다시 wakeup시킨다.

```

//GPIO 핀에 대한 interrupt 설정
void inter_open(void){
    int ret;
    int irq;

    printk(KERN_ALERT "Open Module\n");

    gpio_direction_input(IMX_GPIO_NR(1, 11));
    irq = gpio_to_irq(IMX_GPIO_NR(1, 11));
    printk(KERN_ALERT "IRQ Number : %d\n", irq);
    ret = request_irq(irq, (irq_handler_t)inter_handler_home, IRQF_TRIGGER_FALLING, "home", NULL);
    if (ret) {
        printk(KERN_ALERT "Unable to request IRQ for home: %d\n", ret);
        return;
    }

    gpio_direction_input(IMX_GPIO_NR(1, 12));
    irq = gpio_to_irq(IMX_GPIO_NR(1, 12));
    printk(KERN_ALERT "IRQ Number : %d\n", irq);
    ret = request_irq(irq, (irq_handler_t)inter_handler_back, IRQF_TRIGGER_FALLING, "back", NULL);
    if (ret) {
        printk(KERN_ALERT "Unable to request IRQ for back: %d\n", ret);
        return;
    }

    gpio_direction_input(IMX_GPIO_NR(2, 15));
    irq = gpio_to_irq(IMX_GPIO_NR(2, 15));
    printk(KERN_ALERT "IRQ Number : %d\n", irq);
    ret = request_irq(irq, (irq_handler_t)inter_handler_volup, IRQF_TRIGGER_FALLING, "volup", NULL);
    if (ret) {
        printk(KERN_ALERT "Unable to request IRQ for volup: %d\n", ret);
        return;
    }

    gpio_direction_input(IMX_GPIO_NR(5, 14));
    irq = gpio_to_irq(IMX_GPIO_NR(5, 14));
    printk(KERN_ALERT "IRQ Number : %d\n", irq);
    ret = request_irq(irq, (irq_handler_t)inter_handler_voldown, IRQF_TRIGGER_FALLING | IRQF_TRIGGER_RISING, "voldown", NULL);
    if (ret) {
        printk(KERN_ALERT "Unable to request IRQ for voldown: %d\n", ret);
        return;
    }
}

```

GPIO핀에 대한 interrupt를 설정한다.

```
//interrupt 설정 해제
void inter_release(void){

    free_irq(gpio_to_irq(IMX_GPIO_NR(1, 11)), NULL);

    free_irq(gpio_to_irq(IMX_GPIO_NR(1, 12)), NULL);

    free_irq(gpio_to_irq(IMX_GPIO_NR(2, 15)), NULL);

    free_irq(gpio_to_irq(IMX_GPIO_NR(5, 14)), NULL);

    printk(KERN_ALERT "Release Module\n");
}
```

설정된 interrupt를 해제한다.

```

//home버튼(start)눌렀을 때 동작 READY상태일때만 동작(최초상태거나 volup버튼(RESET) 누르면 READY상태 )
irqreturn_t inter_handler_home(int irq, void* dev_id, struct pt_regs* reg){

    if(mode!=READY) return 0;
    mode=RUN;
    //del_timer(&my_timer);
    my_timer.expires = get_jiffies_64();
    my_timer.function = timer_handler;
    add_timer(&my_timer);
    return IRQ_HANDLED;
}

//back버튼(pause)눌렀을 때 동작 RUN과 PAUSE상태일때 동작하며 상태를 바꿔가며 동작을 수행
irqreturn_t inter_handler_back(int irq, void* dev_id, struct pt_regs* reg){

    if(mode==READY) return 0;

    else if(mode==PAUSE){ //pause 되어 있던 시간 저장 하고 다시 동작
        acc_pause_time+=current_time-pause_time;
        mode=RUN;
        my_timer.expires=get_jiffies_64()+HZ/10;
        my_timer.function=timer_handler;
        add_timer(&my_timer);
    }
    else if(mode==RUN){ //pause 된 시간 저장하고 일시정지
        mode=PAUSE;
        pause_time=current_time;
        del_timer(&my_timer);
    }
    return IRQ_HANDLED;
}

//volup버튼(reset) 눌렀을 때 동작 모든 시간값 초기화하고 my_wq에 디바이스들(fnd,dot) 초기화하는 작업 추가
irqreturn_t inter_handler_volup(int irq, void* dev_id, struct pt_regs* reg){
    if(mode==READY) return 0;
    current_time=0;
    acc_pause_time=0;
    voldown_time=0;
    print_time=0;
    mode=READY;
    struct work_struct *device_work = (struct work_struct *)kmalloc(sizeof(struct work_struct), GFP_KERNEL);
    if (device_work) {
        INIT_WORK((struct work_struct *)device_work, my_wq_device_function);
        queue_work(my_wq, (struct work_struct *)device_work);
    }

    del_timer(&my_timer);
    return IRQ_HANDLED;
}

//voldown버튼(stop) 버튼이 눌렀는지 안눌렀는지를 체크해서 pause와 check_voldown_press값을 설정
irqreturn_t inter_handler_voldown(int irq, void* dev_id, struct pt_regs* reg){

    pause=gpio_get_value(IMX_GPIO_NR(5, 14));

    if(pause!=1){
        //버튼이 눌리면 voldown_time을 현재시간으로 설정
        check_voldown_press=0;
        voldown_time=current_time;
    }
    else check_voldown_press=1;

    return IRQ_HANDLED;
}

```

버튼이 눌렀을 때 동작하는 stopwatch의 기능들

```

//workqueue에 등록될 함수 fnd와 dot 출력하는 역할
static void my_wq_device_function(struct work_struct *work){

    fnd_device(print_time);
    dot_device(print_time);

}

//workqueue에 등록될 함수 프로세스를 깨우고 stopwatch를 종료시키는 역할
static void my_wq_end_function(struct work_struct *work){
    del_timer(&my_timer);
    wake_up_interruptible(&waitq);
}

```

Bottom half routine을 위한 함수들이다 뒤에 있을 timer_handler에서 work queue에 등록되어 실행하게끔 한다.

```

void timer_handler(unsigned long data){ //timer_handler

    current_time++;

    print_time=current_time-acc_pause_time;
    if(mode==PAUSE) acc_pause_time++;

    //fnd,dot출력 작업 workqueue 이용
    struct work_struct *device_work = (struct work_struct *)kmalloc(sizeof(struct work_struct), GFP_KERNEL);
    if (device_work) {
        INIT_WORK((struct work_struct *)device_work, my_wq_device_function);
        queue_work(my_wq, (struct work_struct *)device_work);
    }

    //버튼눌린지 3초가 되면
    if(!pause&&!check_voldown_press&& current_time-voldown_time>=30){

        //종료 작업 workqueue이용
        struct work_struct *end_work=(struct work_struct*)kmalloc(sizeof(struct work_struct),GFP_KERNEL);
        if(end_work){
            INIT_WORK((struct work_struct *)end_work, my_wq_end_function);
            queue_work(my_wq, (struct work_struct *)end_work);
        }
    }

    //0.1초마다 갱신
    my_timer.expires=get_jiffies_64()+HZ/10;
    my_timer.function=timer_handler;
    add_timer(&my_timer);
}

```

Timer_handler이다. Current_time을 이용해서 Top half routine을 담당한다.

-device.c

```
int fnd_device(unsigned long time){ //time 넘겨받아 분과 초 표시 소수점은 제외

    unsigned char value[4]={0,};
    unsigned short int value_short=0;
    int num;
    int m,s;

    num=time/10; //time은 0.1초당 1씩 올라가기 때문에 10으로 나눠서 초를 num에 넣음
    m=num/60; //분
    s=num%60; //초

    if(m<10){ //분이 한자리 수일때
        value[0]=0;
        value[1]=m;
    }
    else{ //분이 두자리 수일때
        value[0]=m/10;
        value[1]=m%10;
    }
    value[2]=s/10;
    value[3]=s%10;

    value_short=value[0] << 12 | value[1] << 8 | value[2] << 4 | value[3];
    outw(value_short,(unsigned int)iom_fpga_addr[FND]);

    return 0;
}

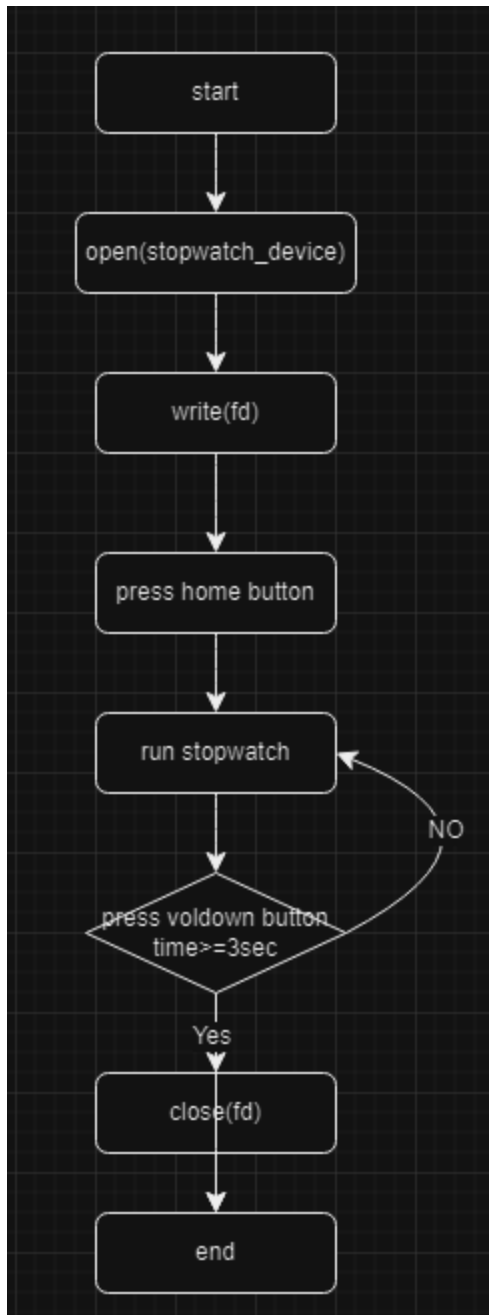
void dot_device(unsigned long time){ //num을 dot에 출력 만약 -1입력시 빈칸 출력

    int i=0;
    int num;
    unsigned short _s_value;
    if(time==-1){
        for(i=0;i<10;i++){
            _s_value=fpga_set_blank[i] & 0x7F;
            outw(_s_value,(unsigned int)iom_fpga_addr[DOT]+i*2);
        }

        return;
    }
    num=time%10; //time은 0.1초당 1씩 증가하는데 dot에서는 0.1초단위를 표현하기 위해 1의자리수만 남김
    for(i=0;i<10;i++){
        _s_value=fpga_number[num][i] & 0x7F;
        outw(_s_value,(unsigned int)iom_fpga_addr[DOT]+i*2);
    }
    return;
}
```

Fnd와 dot에 시간을 출력하는 함수이다.

-app.c



IV. 연구 결과

Home, BACK, VOLUP, VOLDOWN버튼을 누를 때 각각 start, pause, reset, stop을 구현하게끔 했다. Start는 처음과 reset버튼을 눌렀을 때만 동작하도록 구현했고 pause로 일시정지 되었을 때는 다시 pause로만 동작하도록 구현했다. 3초이상 stop버튼을 누르면 종료되도록 하였다.

V. 기타

과제2에서 구현했던 기능과 개념들에 interrupt를 추가로 구현해서 완성하는 과제였는데 처음에는 너무 어렵게 생각해서 현재 시간을 get_jiffies_64()로 표현 하려다가 생각대로 출력이 안되서 시간이 좀 걸렸고 그 후에 따로 변수를 선언해서 시간을 출력했다. Interrupt를 처음 구현했는데 개념을 더 잘 이해하게 되었다.